

Server load factor: structure and the capacity decision

Generated 2026-09-17 15:26 by `sequence_report.py` from `sample_load.csv` (column `load_factor`, timestamps `timestamp`). Seed 2025. Every number below is reproducible from the CSV, this script and the settings listed at the end.

Summary in plain words

What was analysed. 17,280 values of *load factor*, one every 15 min, from 2025-01-01 00:00 to 2025-06-29 23:45. The last 54 days (5,184 values) were locked away before any analysis and used only once, at the end, to check whether anything found earlier holds up on data it had never seen.

The obvious pattern. The value follows a regular cycle (time-of-day, 96 slots). That cycle is treated as known and removed first, so everything below is about what remains *after* the cycle.

Is there a pattern beyond that? Yes. The recent past says something about the value 1 step ahead: a simple model that looks at the last 3 values predicts it 12% better (in squared error) than just using the typical value for that time, and it did so consistently in every stretch of the data, and again on the locked-away data (13% better).

Does the risk depend on the time of the cycle? Yes. On the development data the event rate rises above the break-even chance (5%) only in 10:45–11:15 (6.0%, locked-away data 1.9%) — 2% of the cycle — with the highest risk around 11:00 (5.6%) and the lowest around 01:00 (0.0%). A fixed schedule that adds capacity only in those windows is therefore a candidate: on the locked-away data it cost 0.2986 per step (17.7% more than the current rule) and handled 2 of 74 events.

Is it worth acting on? That was tested on the locked-away data with the costs attached (acting costs 1, a missed event costs 20; the event is: value above 0.86). The current rule cost 0.2537 per step. Using the forecast cost 0.2392 (5.7% less), and simply moving the current rule's trigger level from 0.8 to 0.7799 cost 0.224 (11.7% less). Each difference comes with a range of plausible values; a saving counts only if its whole range is on the saving side.

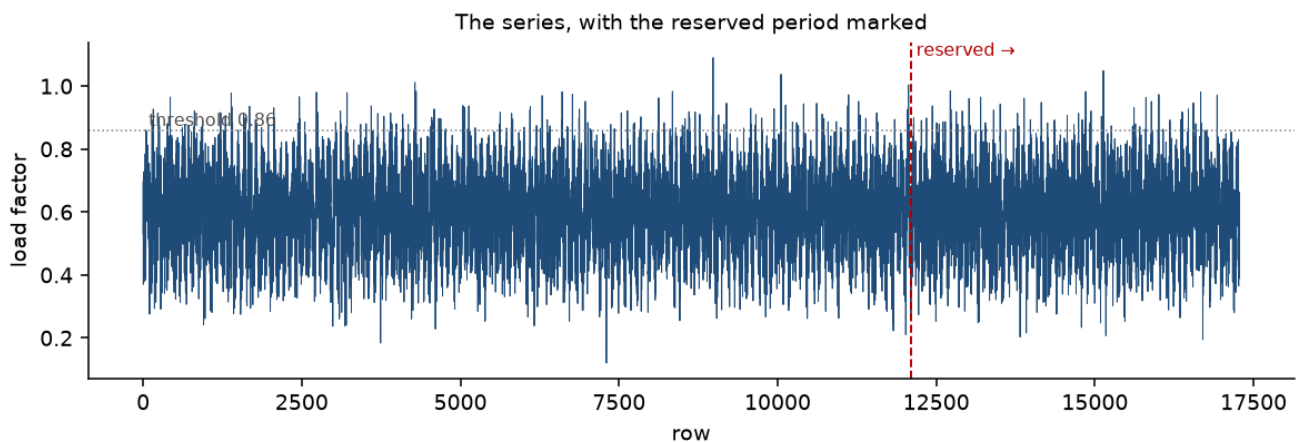
When to scale back down? This report assumes a change of capacity costs nothing (`--cost-switch 0`). Under that assumption capacity can be released as soon as the trigger condition no longer holds; a separate, lower release level only pays when each change costs something. Checked anyway: the best two-level rule found on the development data would scale up when the value is above 0.8009, scale back down when it is below 0.7799. On the locked-away data it cost 0.2531 per step against 0.224 for the single threshold (13.0% more, plausible range [+0.0041, +0.0542]), so no saving from the lower release level is shown. If scaling up or down does cost something (start-up time, migration, a fee), re-run with `--cost-switch` and this comparison becomes the deciding one.

What to do. Adopt the re-tuned threshold (0.7799): its paired interval against the existing rule excludes zero. Not shown to beat the existing rule: the two-level rule (up 0.8009, down 0.7799), the fixed schedule, the forecast policy; keep as candidates and re-confirm later.

1. Series and data checks

check	result
rows	17,280
period	2025-01-01 00:00 to 2025-06-29 23:45
sampling interval	15 min
transform	none (levels analysed as given)
gaps / irregular steps	0
missing values	0
timestamps strictly increasing	yes
known cycle used	time-of-day, 96 slots
clusters for the intervals	calendar days
development	2025-01-01 00:00 to 2025-05-06 23:45 (12,096 rows, 126 clusters)
reserved	2025-05-07 00:00 to 2025-06-29 23:45 (5,184 rows, 54 clusters), untouched until the final evaluation

In plain words. Before any statistics, the data itself is checked: are the readings evenly spaced in time, are any missing, is anything out of order? Each of those creates false patterns. "Development" is the part of the data used to look for patterns and fit models; "reserved" is the part kept back so that whatever is found can be tested on data that played no part in finding it. Without that split, almost anything looks like a pattern. "Clusters" are the blocks (usually days) treated as independent when the uncertainty of a result is computed.



2. Decision

At each step, act for the step 1 ahead if the event — **value above 0.86** — is likely. Acting costs 1; an unhandled event costs 20; so acting pays when the event probability exceeds $1/20 = 5\%$. **Existing rule (B):** act when the current value is above 0.8. **Proposed (C):** act when the forecast event probability exceeds 5%. References: never act (A); the existing rule re-tuned on the development period (B', threshold 0.7799); a two-level rule (H) re-tuned on the development period with a separate release level: scale up when the value is above 0.8009, scale back down when it is below 0.7799. Each change of state costs 0 (changes are free; a release level below the trigger can then only help by chance).

Assumed: acting prevents the whole penalty for that step, is available in time, carries no start-up or switching cost, and is paid for whether or not the event occurs; each step's decision stands alone. Event rate: development 1.6%, reserved 1.4%.

In plain words. The decision is a bet. Acting costs a little every time; not acting costs a lot when the event happens. Dividing the two costs gives the break-even chance: act whenever the event is more likely than 5%. Four ways of deciding are compared: do nothing (A), the rule in use today (B), the same rule with its trigger level re-chosen on past data (B'), and acting on a forecast (C). B' is there to answer a fair question: is a forecast better than merely adjusting the number in the existing rule?

3. Known structure and baselines (development, rolling origin, 5 folds, horizon 1)

baseline / model	skill (1 - MSE/MSE_baseline)
linear AR(3) on the deseasonalised value, vs the calendar-mean baseline	+0.121 (per fold: +0.09, +0.13, +0.12, +0.13, +0.12)
persistence (last value known at decision time)	-0.304
seasonal persistence (same slot one cycle earlier)	-1.001

Calendar means (time-of-day, 96 slots) and coefficients are estimated inside each training window, never from the fold being scored. All folds positive: the stability check passes.

In plain words. "Skill" measures how much better a forecast is than a naive one; 0 means no better, positive means better, negative means worse. The naive forecast here is the typical value for that time of the cycle. "Persistence" (assume the future value equals the latest known one) scores -0.30, and "seasonal persistence" (assume it equals the value one cycle ago) scores -1.00. The simple model scores +0.12. "Rolling origin" means the model was always fitted on earlier data and scored on later data, 5 times over successive stretches (the numbers in brackets); if they are all positive the improvement is steady rather than a lucky patch.

4. Diagnostics (development, deseasonalised)

diagnostic	statistic	p	null	reading
Ljung–Box Q(10), deseasonalised	Q = 1669.4	$p < 0.002$	exchangeability (permutation, B = 500)	not exchangeable; low-order autocorrelation
Rolling-origin kNN forecast skill, horizon 1	skill = +0.086	$p < 0.010$	exchangeable given the clock (permutation within calendar strata; pipeline rebuilt), B = 100	the past carries information about the value 1 step ahead, and the forecaster beats the baseline
CUSUM level (LRV-scaled)	T = 0.97	$p = 0.303$ (analytic)	stationary, weakly dependent, constant mean	no level shift detected (does not establish stationarity)
CUSUM scale ($\sqrt{n}$ bandwidth) — exploratory	T = 0.56	$p = 0.909$ (analytic)	same, on squared deviations; size $\approx 2 \times$ nominal under persistent clustering	no scale change detected

Read each p-value against its own null. Rejections say the sequence is not exchangeable / not stationary / informative about its future; none says the information is worth acting on. Non-rejection by the change tests does not establish stationarity. The scale test is exploratory under persistent variance clustering (size about twice nominal).

In plain words. Each diagnostic asks one question and answers it with a p-value: the chance of seeing a result at least this strong if the answer were "no". A p-value below 0.05 is the usual bar for "this is unlikely to be chance"; " $p \leq 0.002$ " means the result was stronger than every one of the 500 shuffled versions of the data the test compared it with. The first two rows ask whether the order of the values matters (Ljung–Box: do neighbouring values move together? kNN forecast: does the recent past help predict the future, beyond the cycle?). The last two rows ask whether the series changed character part-way through, in its typical level or in how much it varies; a change like that would make any pattern found earlier untrustworthy. "No level shift detected" means the test did not find one, not that none exists.

5. Probability check

Forecast 1 step ahead = calendar mean + AR(3) prediction; residual spread on development 0.09845; event probability from the Gaussian tail (value above 0.86). Reliability is checked **out of fold** on development (7,256 steps whose probability came from a model that never saw them) and on the reserved period. Bins are built around the break-even probability (5%): half of it, it, twice, four times. Intervals are cluster-robust (calendar days: sums of outcome – predicted probability per cluster, variance across clusters; clusters treated as independent), which allows for dependence between consecutive predictions and unequal probabilities inside a bin.

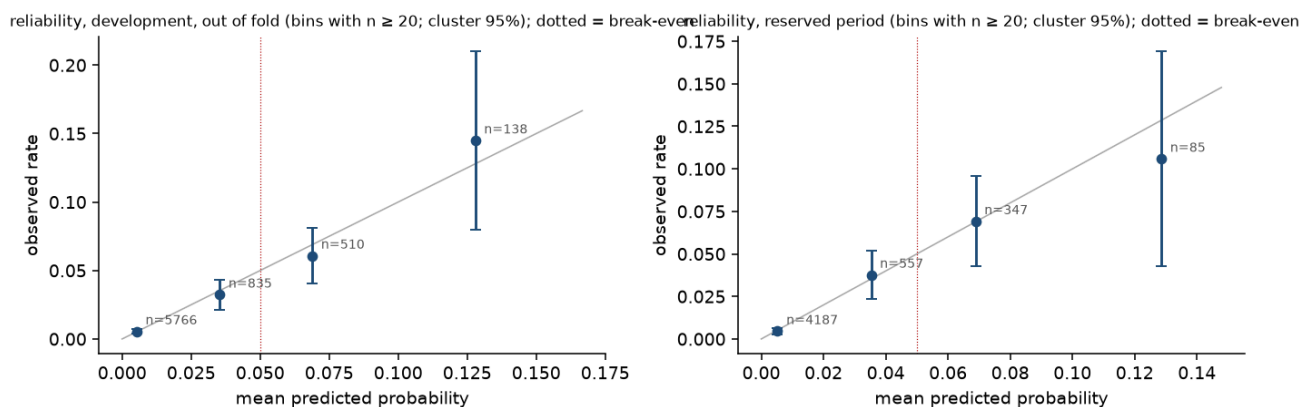
Development, out of fold

bin	n	mean predicted	observed	observed – predicted	cluster 95%
[0%, 2.5%)	5,766	0.5%	0.6%	+0.0 pts	[-0.1, +0.2]
[2.5%, 5%)	835	3.5%	3.2%	-0.3 pts	[-1.4, +0.8]
[5%, 10%)	510	6.9%	6.1%	-0.8 pts	[-2.8, +1.2]
[10%, 20%)	138	12.8%	14.5%	+1.7 pts	[-4.8, +8.2]
[20%, 100%]	7	23.8%	0.0%	-23.8 pts	[-29.5, -18.2]

Reserved period (model frozen at the split)

bin	n	mean predicted	observed	observed – predicted	cluster 95%
[0%, 2.5%)	4,187	0.5%	0.5%	-0.1 pts	[-0.3, +0.1]
[2.5%, 5%)	557	3.5%	3.8%	+0.2 pts	[-1.2, +1.7]
[5%, 10%)	347	6.9%	6.9%	+0.0 pts	[-2.6, +2.7]
[10%, 20%)	85	12.9%	10.6%	-2.3 pts	[-8.6, +4.1]
[20%, 100%]	8	22.6%	12.5%	-10.1 pts	[-36.8, +16.6]

Steps flagged above 5% on the reserved period: 440 (8.5%); observed event rate among them 7.7% against a mean prediction of 8.3%; observed – predicted -0.6 pts, cluster 95% [-2.9, +1.7]. Forecast skill on the reserved period, frozen at the split: +0.128.

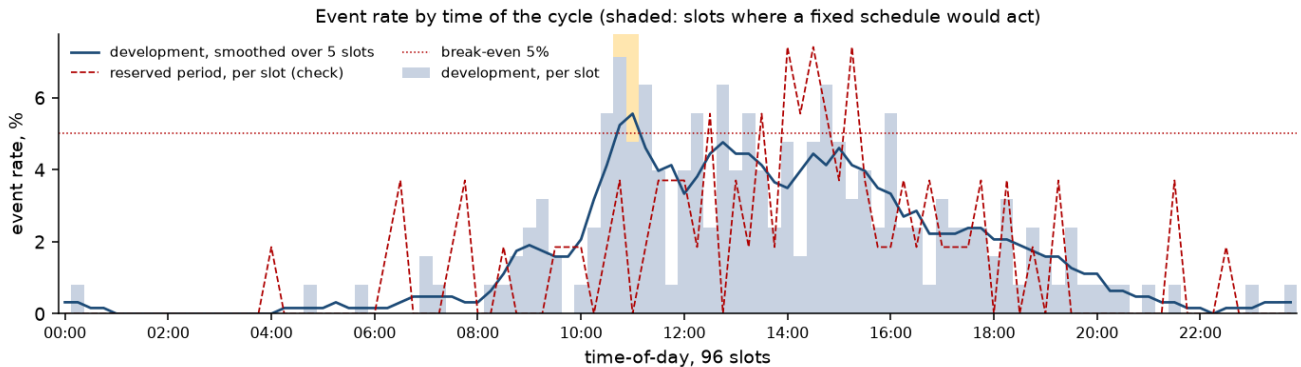


In plain words. The forecast does not say "an event will happen"; it gives a chance. This section checks whether those chances are honest: among all the steps where the forecast said "about 7%", did an event actually happen about 7% of the time? Each row groups steps by their predicted chance and compares the prediction with what happened. "Observed – predicted" near zero means honest; the range next to it says how far off the truth could plausibly be given how few steps the row holds. The rows that matter for the decision are the ones above the break-even chance, and they are usually the thinnest.

6. Event risk by time of the cycle (development)

Event rate per slot of the cycle (time-of-day, 96 slots) on the development period, smoothed over 5 neighbouring slots; a fixed schedule **S** acts in every slot whose smoothed rate exceeds the break-even chance (5%). Highest risk around 11:00 (5.6%), lowest around 01:00 (0.0%); the schedule covers 2% of the cycle. The development period holds 193 events spread over 96 slots (2.0 per slot on average), so per-slot rates are noisy; the smoothing and the reserved-period check guard against reading noise as timing. The reserved-period rates are shown only as a check; the schedule was fixed before they were looked at.

window (schedule acts)	slots	development event rate	n	reserved event rate (check)	n
10:45–11:15	2	6.0%	252	1.9%	108



In plain words. This asks a simpler question than forecasting: is the trouble tied to the clock? If events pile up in the same part of the day (or week) every cycle, then adding capacity on a fixed timetable — no forecast, no threshold, just "more from 11:00 to 17:00" — is a candidate policy, and it is evaluated below alongside the others on the locked-away data. The shaded slots are where the schedule would act; the dashed line is what actually happened in the locked-away period, a check that the timing held up. Rates in the ups and downs of the cycle that never reach the break-even chance do not justify scheduled capacity on their own, however visible the cycle is.

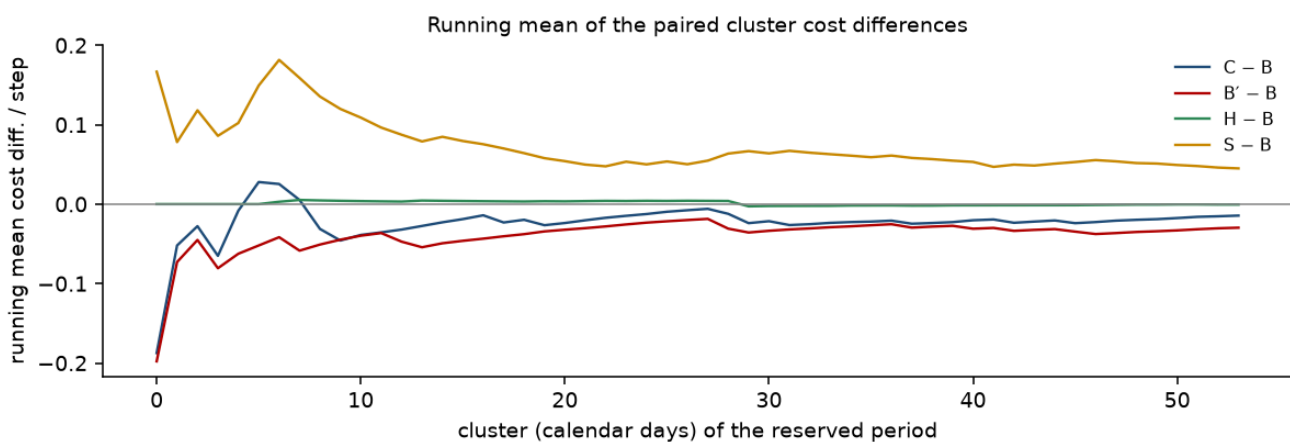
7. Decision evaluation (reserved period; everything fixed from development)

policy	cost / step	actions per day	changes of state per day	events handled
A never act	0.2855	0.00	0.00	0 / 74
B existing rule (0.8)	0.2537	4.72	7.41	21 / 74
B' re-tuned rule (0.7799)	0.224	6.69	9.56	34 / 74
H two-level rule (up 0.8009, down 0.7799)	0.2531	5.04	7.15	22 / 74
S fixed schedule (2% of the cycle)	0.2986	2.00	2.00	2 / 74
C forecast probability > 5%	0.2392	8.15	9.81	34 / 74

Development costs per step, for reference (in-sample for the fitted model): A 0.3192, B 0.266, B' 0.2635, H 0.2611, S 0.3152, C 0.2649.

comparison	mean cost difference / step	relative to the second policy	95% t-interval (paired clusters)	lag-1 autocorr. of cluster diffs	Newey–West (2 lags)
B' – B (the threshold change itself)	-0.02971	-11.7%	[-0.0553, -0.0041]	+0.070	[-0.0543, -0.0052]
H – B (the release level added)	-0.0005787	-0.2%	[-0.0083, +0.0071]	-0.031	[-0.0079, +0.0067]
S – B (the schedule against the existing rule)	+0.04495	17.7%	[+0.0128, +0.0771]	+0.240	[+0.0077, +0.0822]
C – B	-0.01447	-5.7%	[-0.0463, +0.0173]	+0.115	[-0.0469, +0.0179]
H – B' (is the release level worth anything?)	+0.02913	13.0%	[+0.0041, +0.0542]	-0.055	[+0.0069, +0.0514]
S – B'	+0.07465	33.3%	[+0.0288, +0.1205]	+0.230	[+0.0225, +0.1268]
S – H	+0.04552	18.0%	[+0.0115, +0.0795]	+0.244	[+0.0056, +0.0854]
C – B'	+0.01524	6.8%	[-0.0138, +0.0443]	+0.115	[-0.0160, +0.0465]
C – H	-0.01389	-5.5%	[-0.0432, +0.0154]	+0.096	[-0.0430, +0.0153]
C – S	-0.05941	-19.9%	[-0.1048, -0.0140]	+0.124	[-0.1085, -0.0103]

Paired differences over 54 reserved clusters (calendar days); clusters are treated as independent (check the lag-1 autocorrelation and the Newey–West interval, which adjusts for serial dependence). A negative difference favours the first policy. An interval that includes zero means undecided, not equivalent.



In plain words. Each policy was run on the locked-away data exactly as it would have been run live, with nothing adjusted, and its total bill divided by the number of steps. "Actions" is how often it would have acted; "changes of state" how often it switched between acting and not acting (each one is charged when a switching cost is given); "events handled" how many of the events it would have acted on in time. The second table asks whether the differences between policies are real or luck: the bill is compared cluster by cluster (usually day by day), and the range shows where the true average difference plausibly lies. A negative number favours the first-named policy. If the range straddles zero, the data were too few or too variable to tell the two policies apart, which is not the same as saying they are equal. The last two columns check that consecutive clusters are not moving together; the Newey–West range corrects for that and should be close to the first.

8. Reading and action

Structure. Structure detected on the development period: not exchangeable; low-order autocorrelation; the past carries information about the value 1 step ahead, and the forecaster beats the baseline. Rolling-origin skill

+0.121, stable across folds; on the reserved period, frozen at the split, +0.128.

Value. Adopt the re-tuned threshold (0.7799): its paired interval against the existing rule excludes zero. Not shown to beat the existing rule: the two-level rule (up 0.8009, down 0.7799), the fixed schedule, the forecast policy; keep as candidates and re-confirm later.

Rules the reading follows

rule	as applied
a pattern counts as usable	rolling-origin skill ≥ 0.01 and positive in every fold, and the series is not level-like
a fixed schedule (S) acts	in every slot of the cycle whose development event rate, smoothed over neighbouring slots, exceeds the break-even chance; it is a candidate only if it acts somewhere
the two-level rule (H)	both levels chosen on the development data over a 61×61 quantile grid (release level $\leq$ trigger level), with the switching cost included; it is a candidate even when it coincides with B'
candidates are ordered by simplicity	re-tuned threshold, then two-level rule, then fixed schedule, then forecast policy; a more complex candidate is chosen only if it beats the simpler one
a candidate beats a policy	its paired interval against that policy lies entirely below zero
two candidates the data cannot separate	the simpler one is preferred as a practical judgment (undecided $\neq$ equivalent)
a detected pattern on its own	is never a reason to change the policy

Settings used

setting	value
reserve	0.3
lags / horizon	3 / 1
folds / first training window	5 / 0.4
kNN neighbours	25
period	inferred
cluster	calendar days
transform	none
direction	above
costs act / miss / switch	1 / 20.0 / 0
min-skill / level-warn	0.01 / 0.9
B (Ljung–Box / forecast)	500 / 100
seed	2025
delimiter	','

What each number means

term	meaning
rows / period / sampling interval	how many values there are, the dates they cover, and how far apart in time consecutive values are.
transform	whether the values were analysed as given (levels) or as changes from one step to the next; changes are the right input for a quantity that wanders, such as a price or a running total.
gaps, missing values, strictly increasing	problems in the raw data that would create false patterns: skipped or duplicated times, empty cells, values out of order.
development / reserved	the data used to search and fit (development) and the data locked away to test the result once (reserved).
clusters	blocks of consecutive values (usually calendar days) treated as independent when uncertainty is computed; values inside a cluster may be related.
known cycle	a regular daily or weekly rhythm inferred from the timestamps. It is removed before testing, because a rhythm everyone already knows about is not a discovery.
calendar mean / baseline	the typical value for that time of the cycle, learned from earlier data only. Every forecast is judged against it.
horizon	how many steps ahead the forecast and the decision look.
skill	$1 - (\text{the model's squared error} \div \text{the baseline's squared error})$. 0 = no better than the baseline; +0.10 = 10% less squared error; negative = worse.
persistence / seasonal persistence	two naive forecasts: "the future value equals the latest known one" and "it equals the value one cycle ago". Any proposed model must beat them.
rolling origin, per fold	the model is fitted on earlier data and scored on later data, repeatedly, moving forward in time; each stretch is a fold. Consistent positive folds mean a steady effect.
Ljung–Box Q(10)	a single number summarising how strongly each value is related to the previous ten. Larger means stronger relation.
kNN forecast skill	the skill of a simple, flexible forecaster (k nearest neighbours) that uses the last few values; it can pick up patterns a straight-line model misses.
null / permutation / B	the null is the "nothing here" scenario the test compares against, made concrete by shuffling the data B times in a way that destroys exactly the pattern in question and keeps everything else.
p-value	the share of shuffles (or, for analytic tests, the theoretical chance) that did as well as the real data. Below 0.05 is the customary bar; $p \leq 1/(B+1)$ is the smallest value B shuffles can report.
CUSUM level / scale	tests for a change part-way through the series in its typical level, or in how widely it varies. "Exploratory" on the scale test means its p-values are known to be somewhat too small on series whose variability comes in bursts.
event / threshold / direction	what the decision cares about: the value going above, below, or beyond $\pm$ the threshold.
residual spread	how far the forecast typically misses by. Together with the forecast it gives the chance of an event.
bins, observed – predicted, cluster 95%	steps grouped by predicted chance (around the break-even chance); the difference between what happened and what was predicted; and the range in which that difference plausibly lies, computed cluster by cluster.
event risk by time of the cycle / schedule (S)	how often the event happens in each slot of the daily or weekly rhythm, learned on the development data; the schedule adds capacity in the slots where that rate is above the break-even chance, with no forecast involved.
two-level rule (H) / release level / switching cost	scale up when the value passes the trigger level, and scale back down only when it falls below a lower release level; the gap stops the system flapping. The switching cost is what

term	meaning
	each change costs (start-up, migration, a fee); with a switching cost of zero a lower release level can only help by chance.
policies A, B, B', C	never act; the existing rule; the existing rule with its trigger level re-chosen on development data; act when the forecast chance exceeds the break-even chance.
break-even chance	cost of acting $\div$ cost of a missed event. Acting is worth it when the chance of the event is above this.
cost / step, actions, events handled	the average bill per step under a policy; how often it acts; how many events it acted on in time.
paired difference, 95% t-interval	the bill of one policy minus the bill of another, computed for each cluster, then averaged; the interval is the range of plausible averages. If it straddles zero the data cannot say which policy is cheaper.
lag-1 autocorrelation, Newey–West	a check that consecutive clusters are not moving together (which would make the interval too narrow); the Newey–West interval adjusts for that and should be close to the plain one.
seed	the starting point of the random shuffles, fixed so that re-running gives the same numbers.

Runtime 117 s. Diagnostics from randomness_toolkit.py; method and validation in the accompanying article.